PHASE 3 · WEEK 11 · DAY 72

Practice: Regex Exercises

Validation and extraction — putting the re module to work

■ Today's Goals

Day 72 is all about putting Regex into action for real-world backend tasks: **Validation** (checking if data follows a correct format) and **Extraction** (pulling useful pieces of data out of messy strings).

In backend development, you'll use Regex every day to validate user sign-up forms (emails, passwords, phone numbers) before saving data to a database.

- **Validation:** check if user input matches strict rules (e.g., valid email, strong password).
- **Extraction:** pull specific parts of a text (e.g., extract phone numbers or dates from log files).

■■ Key Regex Patterns for Day 72

Task	Regex Pattern	Explanation
Email	`r"^[\w\.-]+@[\w\.-]+\.\w+$"`	Starts with word chars/dots/hyphens, has @, domain, and a TLD.
Phone (PK/US)	`r"^\+?\d{10,14}$"`	Optional +, followed by 10 to 14 digits.
Date (YYYY-MM-DD)	`r"\b\d{4}-\d{2}-\d{2}\b"`	Matches date strings like 2026-08-03.

■ Today's Practice Script: day72.py

Create a new file in VS Code: **`backend/day72.py`** and write the code below:

Part 1 — Validation Functions

```
import re

# ========================================
# PART 1: VALIDATION FUNCTIONS
# ========================================
def validate_email(email):
    # ^ = start of string, $ = end of string
    pattern = r"^[\w\.-]+@[\w\.-]+\.\w+$"
    if re.match(pattern, email):
        return True
    return False

def validate_password(password):
    """
    Password Rules:
    - At least 8 characters
    - At least 1 digit (\d)
    - At least 1 uppercase letter ([A-Z])
    """
    if len(password) < 8:
        return False
    if not re.search(r"\d", password):
        return False
    if not re.search(r"[A-Z]", password):
        return False
    return True
```

Part 2 — Extraction from Log Text

```
# ========================================
# PART 2: EXTRACTION FROM LOG TEXT
# ========================================
log_data = """
[2026-08-01 10:15:30] USER_LOGIN user_id=101 status=SUCCESS
[2026-08-02 14:22:05] ERROR failed connection to server_id=502
[2026-08-03 09:00:12] USER_LOGOUT user_id=101 status=SUCCESS
"""

def extract_dates(text):
    # Extracts all YYYY-MM-DD dates
    date_pattern = r"\b\d{4}-\d{2}-\d{2}\b"
    return re.findall(date_pattern, text)
```

Running the Demo

```
# ========================================
# RUNNING THE DEMO
# ========================================
if __name__ == "__main__":
    print("--- ■ REGEX VALIDATION DEMO ---")

    test_emails = ["shoaib@gmail.com", "invalid-email@", "user.name@domain.co"]
    for email in test_emails:
        is_valid = validate_email(email)
        status = "■ Valid" if is_valid else "■ Invalid"
        print(f"Email: {email:<25} -> {status}")

    print("\n--- ■ PASSWORD VALIDATION DEMO ---")
    test_passwords = ["short", "nodigitshere", "Password123", "lower1234"]
    for pwd in test_passwords:
        is_valid = validate_password(pwd)
        status = "■ Strong" if is_valid else "■ Weak"
        print(f"Password: {pwd:<20} -> {status}")

    print("\n--- ■ DATA EXTRACTION DEMO ---")
    found_dates = extract_dates(log_data)
    print(f"Extracted Log Dates: {found_dates}")
```

■ Your Task Today

- Copy and type out `backend/day72.py` in VS Code.
- Run `python backend/day72.py` in your terminal.
- Look closely at how `re.match()` checks full strings from start to end (`^...$`), while `re.search()` looks anywhere inside the string.
- Share your terminal output when you run it! ■

> **■ Remember**
>
> r"^...$" pins a pattern to the entire string, which is exactly what `re.match()` uses for full-format checks like emails and passwords. `re.search()` and `re.findall()` don't need anchors unless you want to control where in the string the match starts or ends.